

Variable Font Inspector

Specimen tool for auditing variable axes, kerning pairs, and OpenType feature coverage across arbitrary sample text and weights.

TypeScript	OpenType.js	HarfBuzz	Vite
------------	-------------	----------	------

VERSION	STATUS	LICENSE	RUNS	FORMATS	UPDATED
1.1.0	Open Source	MIT	Browser, CLI	TTF, OTF, WOFF2	Sep 2026

1. Overview

The Variable Font Inspector answers questions a specimen page cannot: which axes does this font actually expose and over what ranges, which OpenType features fire on this particular sample text, and which kerning pairs in that text are covered by the font versus falling back to default spacing. It loads a font file locally and reports on it — nothing is uploaded.

Shaping is done with HarfBuzz compiled to WebAssembly, the same engine browsers use, so the report reflects what will actually render rather than an approximation. Table parsing is handled separately by OpenType.js, which means axis and feature metadata can be read even from fonts with defects that would stop shaping.

2. Analysis flow

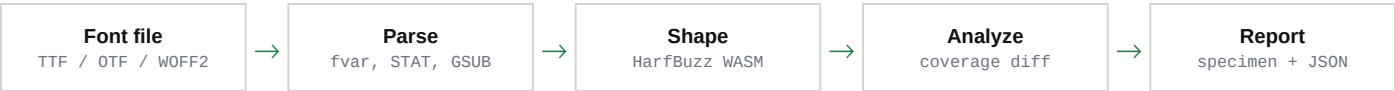


Figure 1 — Sample text is shaped once per axis instance under test; coverage is derived by comparing runs with features on and off.

Feature coverage is measured, not declared. The tool shapes the sample text twice for each feature — once enabled, once disabled — and compares the resulting glyph and position runs. A feature the font advertises but that changes nothing for this text is reported as unexercised, which is usually the more useful fact.

3. Components

Loader	Local file or URL; WOFF2 decompression, checksum validation, and defect reporting.
Table parser	fvar and STAT for axes and named instances, GSUB and GPOS for features and pairs.
Shaper	harfbuzzjs runs with configurable script, language, direction, and feature sets.
Analyzer	Feature-exercised diffing, missing glyph detection, and kern pair coverage over the sample.
Specimen	Live waterfall and axis sliders rendered with the loaded font at any instance.
Reporter	JSON and standalone HTML output, identical from the browser and the CLI.

4. Audit configuration

```
{
  "font": "../dist/Inter-Variable.woff2",
  "sample": "Quartz flock waved; fjord myth VII – “affix” 1/2 No. 42",
  "instances": [
    { "wght": 400 }, { "wght": 700 },
    { "wght": 400, "opsz": 14 }, { "wght": 900, "opsz": 32 }
  ],
  "features": ["kern", "liga", "dlig", "frac", "onum", "ss01", "case"],
  "script": "latn", "language": "ENG",
  "checks": {
    "missingGlyphs": "error",
    "unexercisedFeature": "warn",
    "kernPairCoverage": { "min": 0.90, "level": "warn" },
    "axisRangeMismatch": "error" // fvar vs STAT disagreement
  },
  "report": ["json", "html"]
}
```

Listing 1 — The same configuration drives the interactive view and the CLI used in CI.

5. Checks performed

CHECK	DETECTS	TYPICAL CAUSE
Missing glyphs	Characters in the sample with no glyph.	Incomplete character set for the language.
Axis range mismatch	fvar and STAT disagreeing on an axis.	Hand-edited or subset-damaged tables.
Unexercised feature	Advertised features that change nothing.	Copied feature list, or wrong sample text.
Kern pair gaps	Adjacent pairs with no kerning applied.	Subsetting that dropped GPOS coverage.

6. Using it in CI

- The CLI takes the same configuration file and exits non-zero when a check at error level fires, which makes a font build pipeline fail on a subset that silently dropped kerning.
- JSON reports are stable and diffable between runs, so a pull request can show exactly which pairs or features changed when a font is re-subset or re-exported.
- Nothing leaves the machine in either mode: parsing and shaping run locally in the browser or in Node, which matters for unreleased typefaces under licence restriction.

7. Performance and limits

DIMENSION	MEASURED	NOTES
Full audit, 4 instances	380 ms	1.1 MB variable font, 60-character sample.
Load and parse	90 ms	WOFF2 decompression included.
Sample length ceiling	~8,000 chars	Beyond this the pairwise coverage pass dominates.
Script coverage	Latin, Greek, Cyrillic	Complex scripts shape correctly but lack tailored checks.

8. Roadmap

RELEASE	PLANNED WORK	TARGET
1.2	Two-font comparison mode for before-and-after subsetting review.	Q4 2026
1.3	Tailored checks for Arabic and Devanagari shaping behaviour.	Q1 2027
2.0	Colour font and COLRv1 palette inspection.	Q2 2027